
SmolVLA + Koch v1.1 실습 가이드

Vision-Language-Action 모델을 활용한
로봇팔 제어 입문

RISE 창업동아리

국립목포해양대학교

2026년 4월

대상 및 구성

SmolVLA를 처음 접하는 대학생을 위한 단계별 튜토리얼입니다.

Part A (3-6장): **로봇 없이** PC만으로 실습 가능

Part B (7-9장): Koch v1.1 로봇팔을 사용한 실습

소스코드는 src/ 폴더에 별도 파일로 제공됩니다.

Contents

1 SmolVLA란 무엇인가?	3
1.1 개요	3
1.2 아키텍처	3
1.3 왜 SmolVLA인가?	4
2 사전 준비 및 설치	5
2.1 소프트웨어 요구사항	5
2.2 설치	5
2.3 설치 확인	5
2.4 Windows 11에서 설치	5
2.4.1 Step 1: WSL2 설치	5
2.4.2 Step 2: WSL2 내부에서 사전 패키지 설치	6
2.4.3 Step 3: NVIDIA GPU 드라이버 (CUDA on WSL2)	6
2.4.4 Step 4: GUI 표시 설정 (시각화용)	6
2.4.5 Windows 네이티브 (WSL2 없이) 사용 시 참고	7
3 공개 데이터셋 탐색	8
3.1 사용 가능한 공개 데이터셋	8
3.2 데이터셋 시각화 (CLI)	8
3.3 Python으로 데이터셋 탐색	8
4 시뮬레이션에서 정책 학습	10
4.1 지원 시뮬레이션 환경	10
4.2 CLI로 학습 (가장 간단)	10
4.3 Python API로 학습	10
4.4 시뮬레이션에서 평가	11
5 로봇 없이 모델 추론 테스트	12
5.1 데이터셋 프레임에서 추론	12
5.2 사전학습 모델 목록	12
5.3 Google Colab 노트북	12
6 Koch v1.1 로봇팔 설정	13
6.1 하드웨어 구성	13
6.2 Step 1: USB 포트 확인	13
6.3 Step 2: 모터 ID 설정 (최초 1회)	14
6.4 Step 3: 캘리브레이션	14
6.5 Step 4: 텔레오퍼레이션 테스트	14
7 데이터 수집 및 Fine-tuning	16
7.1 시연 데이터 수집	16
7.1.1 CLI로 수집	16
7.1.2 Python API로 수집	16
7.2 SmolVLA Fine-tuning	16
7.2.1 CLI로 Fine-tuning (권장)	16
7.2.2 Python API로 Fine-tuning	17
7.3 주요 설정 파라미터	17
8 추론 및 평가	18
8.1 CLI로 실행	18
8.2 Python API로 실행	18

9 전체 워크플로우 요약	19
9.1 소스코드 파일 목록	19
10 트러블슈팅	20
11 참고 자료	20
11.1 인용	21

1 SmolVLA란 무엇인가?

1.1 개요

SmolVLA(Small Vision-Language-Action)는 HuggingFace LeRobot 팀이 개발한 **소형 VLA 모델**로, 저비용 로봇 하드웨어에서도 효율적으로 동작하도록 설계되었다. 2025년 6월 arXiv 논문¹으로 공개되었으며, Apache 2.0 라이선스로 누구나 사용할 수 있다.

VLA 모델이란?

Vision-Language-Action (VLA) 모델은 카메라 영상(Vision)과 자연어 명령(Language)을 입력받아 로봇의 행동(Action)을 직접 출력하는 end-to-end 모델이다. 기존의 rule-based 제어와 달리, 시연(demonstration) 데이터로부터 학습하여 다양한 작업을 수행할 수 있다.

1.2 아키텍처

SmolVLA의 핵심 구조는 다음과 같다:

- **VLM Backbone:** SmolVLM2-500M-Video-Instruct (~500M 파라미터)
- **Action Expert:** Cross-attention 기반 별도 행동 생성 헤드
- **학습 방법:** Flow Matching (자기회귀 토큰 예측이 아닌 연속 행동 생성)
- **입력:** 다중 카메라 영상 (최대 3개) + 로봇 상태(proprioception) + 언어 명령
- **출력:** 연속 행동 벡터 (기본 50 스텝 chunk)

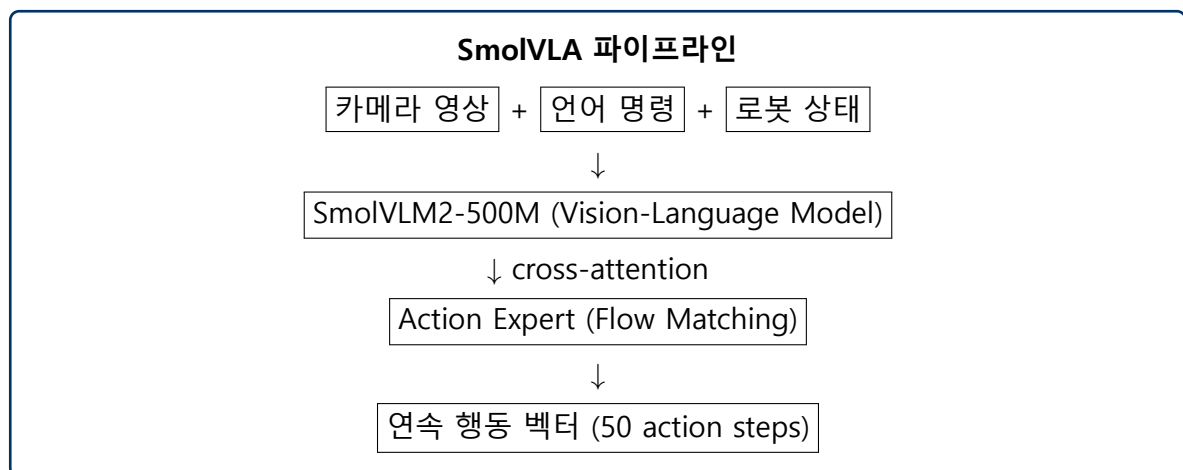


Figure 1: SmolVLA 모델 구조 개요

¹Shukor et al., "SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics," arXiv:2506.01844, 2025.

1.3 왜 SmolVLA인가?

Table 1: VLA 모델 비교

모델	파라미터	학습 GPU	특징
RT-2 (Google)	55B	TPU Pod	대규모, 비공개
OpenVLA	7B	8×A100	오픈소스, 고사양 필요
π_0 (Physical Intelligence)	3B	Multi-GPU	비공개
SmolVLA	~500M	단일 GPU	오픈소스, 저비용, 고효율

2 사전 준비 및 설치

2.1 소프트웨어 요구사항

Table 2: 소프트웨어 요구사항

항목	최소 사양	권장 사양
GPU	NVIDIA 8GB VRAM (추론)	RTX 3090/4090 24GB (학습)
RAM	16GB	32GB
OS	Ubuntu 22.04 / Windows 11 (WSL2)	Ubuntu 22.04
Python	3.12+	3.12
CUDA	12.0+	12.4

2.2 설치

가상환경 생성 및 패키지 설치

```
# 가상환경 생성 & 활성화
python3.12 -m venv ~/smolvla-env
source ~/smolvla-env/bin/activate
pip install --upgrade pip

# === 로봇 없이 시뮬레이션만 실습할 경우 ===
pip install "lerobot[smolvla,pusht]"

# === Koch v1.1 로봇팔도 사용할 경우 ===
pip install "lerobot[smolvla,dynamixel,pusht]"

# HuggingFace 로그인
pip install huggingface_hub
huggingface-cli login
```

2.3 설치 확인

```
src/01_test_install.py
```

```
python src/01_test_install.py
```

정상적으로 설치되었다면 LeRobot 버전, GPU 정보, SmolVLAPolicy 임포트 성공 메시지가 출력된다.

2.4 Windows 11에서 설치

Windows 11에서는 **WSL2 (Windows Subsystem for Linux 2)**를 사용하여 Linux 환경을 구성한 뒤 실습을 진행한다. LeRobot과 SmolVLA는 Linux 기반으로 개발되었으므로, 네이티브 Windows에서는 일부 패키지(특히 시뮬레이션 환경, 로봇 제어)가 정상 동작하지 않을 수 있다.

2.4.1 Step 1: WSL2 설치

PowerShell을 **관리자 권한**으로 실행하여 WSL2를 설치한다.

PowerShell (관리자 권한)에서 WSL2 설치

```
# WSL2 설치 (Ubuntu 22.04 기본)
wsl --install

# 이미 설치된 경우 Ubuntu 22.04 지정
wsl --install -d Ubuntu-22.04

# 설치 후 PC 재부팅
```

재부팅 후 시작 메뉴에서 Ubuntu를 실행하면 초기 사용자 이름과 비밀번호를 설정하는 화면이 나타난다.

2.4.2 Step 2: WSL2 내부에서 사전 패키지 설치

Ubuntu 터미널이 열리면 아래 명령으로 시스템 패키지를 먼저 설치한다.

WSL2 Ubuntu 내부에서 실행

```
# 시스템 패키지 업데이트
sudo apt update && sudo apt upgrade -y

# 필수 도구 설치
sudo apt install -y python3.12 python3.12-venv python3-pip \
    git ffmpeg libgl1-mesa-glx
```

이후 가상환경 생성, LeRobot 패키지 설치, HuggingFace 로그인 등은 **섹션 2.2의 설치 과정과 동일**하다. WSL2는 실제 Linux 커널을 실행하므로, 위 사전 패키지 설치 이후에는 본 가이드의 모든 Linux 명령어가 그대로 동작한다.

2.4.3 Step 3: NVIDIA GPU 드라이버 (CUDA on WSL2)

WSL2에서 GPU를 사용하려면 **Windows 측**에 최신 NVIDIA 드라이버가 설치되어 있어야 한다. WSL2 내부에는 별도의 드라이버를 설치하지 않는다.

GPU 확인 (WSL2 Ubuntu 내부)

```
# 가GPU 정상 인식되는지 확인
nvidia-smi
```

Windows NVIDIA 드라이버 요구사항

- **Windows에서** NVIDIA 드라이버 버전 **525.60 이상** 필요
- NVIDIA 공식 사이트에서 **Game Ready / Studio 드라이버**를 설치한다
- WSL2 내부에서는 `sudo apt install nvidia-driver-*`를 실행하지 **않는다**
- CUDA Toolkit은 WSL2 내부에서 필요 시 설치:
`sudo apt install nvidia-cuda-toolkit` 또는 PyTorch에 포함된 CUDA 사용

2.4.4 Step 4: GUI 표시 설정 (시각화용)

데이터셋 시각화나 시뮬레이션 렌더링 등 GUI가 필요한 경우 아래 설정이 필요하다.

WSL2 GUI 설정

```
# Windows 은11 가WSLg 기본 내장되어 GUI 앱이 바로 실행됨
# 확인: 아래 명령으로 간단한 GUI 테스트
```

```
sudo apt install -y x11-apps
xclock # 시계 창이 뜨면 정상
```

WSLg가 동작하지 않을 경우

Windows 11 22H2 이상에서는 WSLg가 기본 지원된다. 만약 GUI가 표시되지 않으면:

1. `wsl --update` 명령으로 WSL을 최신 버전으로 업데이트한다
2. Windows 업데이트를 최신 상태로 유지한다
3. PC를 재부팅한다

2.4.5 Windows 네이티브 (WSL2 없이) 사용 시 참고

WSL2 없이 Windows에서 직접 실습하는 것은 **권장하지 않으나**, Part A (로봇 없이 실습) 중 일부 스크립트는 실행 가능하다.

Windows PowerShell에서 설치 (제한적)

```
# Python 3.12 설치 후 (python.에서org 다운로드)
python -m venv C:\smolvla-env
C:\smolvla-env\Scripts\activate

pip install --upgrade pip
pip install "lerobot[smolvla]"

# 주의: pusht 시뮬레이션 환경은 에서Windows 설치 실패할 수 있음
# 주의: dynamixel로봇( 제어)은 Linux 전용 의존성이 있을 수 있음
```

Windows 네이티브 제한사항

- gym-pusht 등 시뮬레이션 환경이 설치되지 않을 수 있다
- 로봇 제어(dynamixel)는 Linux 시리얼 라이브러리에 의존하므로 동작하지 않을 수 있다
- **Part B (로봇팔 실습)는 반드시 WSL2 또는 네이티브 Linux에서 진행한다**
- 데이터셋 탐색, 모델 추론 등 **Part A의 일부**는 Windows에서도 가능하다

Part A: 로봇 없이 실습

이 파트는 PC(또는 노트북)만 있으면 실행할 수 있습니다.
로봇팔이 없어도 SmolVLA의 핵심 개념을 체험할 수 있습니다.

3 공개 데이터셋 탐색

HuggingFace Hub에는 LeRobot 팀이 공개한 다양한 시뮬레이션/실제 로봇 데이터셋이 있다. 로봇 없이도 이 데이터를 다운로드하여 구조를 파악하고, 모델 학습에 활용할 수 있다.

3.1 사용 가능한 공개 데이터셋

Table 3: 주요 공개 데이터셋 (로봇 불필요)

데이터셋 ID	환경	Action 차원	설명
lerobot/pusht	PushT (2D)	2	T자 블록 밀기, 가장 가벼움
lerobot/pusht_image	PushT (2D)	2	이미지 관측 버전
lerobot/aloha_sim_transfer_cube_human	ALOHA sim	14	양팔 로봇 큐브 전달
lerobot/aloha_sim_insertion_human	ALOHA sim	14	양팔 로봇 삽입 작업

3.2 데이터셋 시각화 (CLI)

데이터셋 시각화 (Rerun 뷰어)

```
# 에피소드 0 시각화
lerobot-dataset-viz \
  --repo-id lerobot/pusht \
  --episode-index 0
```

온라인 시각화: https://huggingface.co/spaces/lerobot/visualize_dataset

3.3 Python으로 데이터셋 탐색

src/02_explore_dataset.py

이 스크립트는 PushT 데이터셋을 다운로드하여 메타데이터, 프레임 구조, 이미지를 확인한다.

```
python src/02_explore_dataset.py
```

출력 예시:

```
총
에피소드 수 : 206총
프레임 수 : 25650
FPS : 10카메라
키 : ['observation.image']키

목록: ['observation.image', 'observation.state', 'action', ...]
observation.image | shape=(3, 96, 96) | dtype=torch.float32
observation.state | shape=(2,) | dtype=torch.float32
```

action	shape=(2,)	dtype=torch.float32
--------	------------	---------------------

4 시뮬레이션에서 정책 학습

로봇 없이 시뮬레이션 환경에서 정책(policy)을 학습할 수 있다. 여기서는 가장 가벼운 **PushT** 환경에서 **Diffusion Policy**를 학습하는 방법을 다룬다.

4.1 지원 시뮬레이션 환경

Table 4: LeRobot 시뮬레이션 환경

환경	Action 차원	FPS	설치 옵션	난이도
PushT	2	10	lerobot[pusht]	입문 (권장)
ALOHA sim	14	50	lerobot[aloha]	중급
LIBERO	7	30	lerobot[libero]	고급 (Linux만)

4.2 CLI로 학습 (가장 간단)

PushT에서 Diffusion Policy 학습 (CLI)

```
# 데이터셋은 에서Hub 자동 다운로드됨
lerobot-train \
  --dataset.repo_id=lerobot/pusht \
  --policy.type=diffusion \
  --env.type=pusht \
  --output_dir=outputs/train/diffusion_pusht \
  --batch_size=64 \
  --steps=5000 \
  --eval_freq=2500 \
  --save_freq=2500 \
  --policy.device=cuda
```

빠른 테스트

위 명령은 steps=5000으로 설정하여 빠르게 동작을 확인한다. 실제 성능을 위해서는 steps=100000 이상이 필요하다.

4.3 Python API로 학습

src/03_train_sim_diffusion.py

이 스크립트는 PushT 데이터셋으로 Diffusion Policy를 학습하는 전체 파이프라인을 포함한다:

1. 데이터셋 메타데이터 로드 및 피처 설정
2. 정책 모델 생성 (DiffusionPolicy)
3. 데이터 로더 구성 (temporal windows 포함)
4. 학습 루프 (loss 출력)
5. 체크포인트 저장

```
python src/03_train_sim_diffusion.py
```

4.4 시뮬레이션에서 평가

학습된 정책(또는 Hub의 사전학습 모델)을 시뮬레이션에서 평가한다.

사전학습 모델로 PushT 시뮬레이션 평가

```
# 의Hub 사전학습 모델 평가
lerobot-eval \
  --policy.path=lerobot/diffusion_pusht \
  --env.type=pusht \
  --eval.n_episodes=10 \
  --eval.batch_size=10 \
  --policy.device=cuda

# 직접 학습한 모델 평가
lerobot-eval \
  --policy.path=outputs/train/diffusion_pusht/checkpoints/last/pretrained_model \
  --env.type=pusht \
  --eval.n_episodes=10 \
  --policy.device=cuda
```

평가 결과로 성공률, 에피소드 영상(MP4)이 출력 디렉토리에 저장된다.

5 로봇 없이 모델 추론 테스트

사전학습된 정책 모델을 로드하여, 기존 데이터셋의 프레임에서 행동을 예측하고 ground truth와 비교한다. **로봇이 없어도** 모델의 동작 원리를 이해할 수 있다.

5.1 데이터셋 프레임에서 추론

```
src/04_inference_no_robot.py
```

```
python src/04_inference_no_robot.py
```

출력 예시:

Frame	Predicted Action	Ground Truth
0	[0.234, 0.567, ...]	[0.245, 0.551, ...]
1	[0.241, 0.572, ...]	[0.250, 0.560, ...]
...		

예측값과 GT가 다른 이유

Diffusion Policy는 **확률적(stochastic)** 모델이다. 같은 관측에 대해 여러 유효한 행동이 존재할 수 있으므로, 예측값이 GT와 정확히 일치하지 않는 것은 정상이다. 중요한 것은 예측값이 합리적인 범위 안에 있는지이다.

5.2 사전학습 모델 목록

Hub에서 바로 다운로드하여 테스트할 수 있는 사전학습 모델들이다:

Table 5: Hub에서 사용 가능한 사전학습 정책 모델

모델 ID	환경	정책 유형
lerobot/diffusion_pusht	PushT	Diffusion Policy
lerobot/act_aloha_sim_transfer_cube_human	ALOHA sim	ACT
lerobot/act_aloha_sim_insertion_human	ALOHA sim	ACT
lerobot/smolvla_base	범용 (VLA)	SmolVLA

5.3 Google Colab 노트북

GPU가 없는 경우, Google Colab에서 무료 GPU를 사용하여 실습할 수 있다:

- **ACT 학습 노트북:** <https://colab.research.google.com/github/huggingface/notebooks/blob/main/lerobot/training-act.ipynb>
- **SmolVLA 학습 노트북:** <https://colab.research.google.com/github/huggingface/notebooks/blob/main/lerobot/training-smolvla.ipynb>
- **Robot Learning Tutorial:** <https://huggingface.co/spaces/lerobot/robot-learning-tutorial>

Part B: Koch v1.1 로봇팔 실습

이 파트부터는 **Koch v1.1** 로봇팔 하드웨어가 필요합니다.

6 Koch v1.1 로봇팔 설정

6.1 하드웨어 구성

Table 6: Koch v1.1 하드웨어 구성

부품	설명	수량
Dynamixel XL330/XL430	서보 모터	6개/팔
Waveshare 또는 U2D2	모터 컨트롤러 보드	2개
5V 전원 공급기	모터 전원	2개
USB 케이블	PC-컨트롤러 연결	2개
3D 프린팅 부품	프레임, 그리퍼 등	1세트
USB 웹캠	OpenCV 호환 카메라	1-3개

Koch v1.1 조립 참고

조립 가이드: <https://github.com/jess-moss/koch-v1-1>

LeRobot Koch 문서: <https://huggingface.co/docs/lerobot/koch>

6.2 Step 1: USB 포트 확인

```
lerobot-find-port
```

포트 권한 설정 (Permission denied 오류 시)

```
sudo usermod -aG dialout $USER
# 로그아웃 후 다시 로그인
```

Windows 11 (WSL2)에서 USB 장치 연결

WSL2는 기본적으로 USB 장치에 접근할 수 없다. **usbipd-win**을 사용하여 Windows의 USB 장치를 WSL2에 전달해야 한다.

1단계: Windows에서 usbipd 설치 (PowerShell 관리자 권한)

```
winget install --id=usbipd-win
```

2단계: WSL2 Ubuntu 내부에서 USB 지원 패키지 설치

```
sudo apt install linux-tools-generic hwddata
sudo update-alternatives --install /usr/local/bin/usbip \
usbip /usr/lib/linux-tools/*-generic/usbip 20
```

3단계: USB 장치 연결 (PowerShell 관리자 권한)

```
# 연결된 USB 장치 목록 확인
usbipd list
```

```
# 에 WSL2 USB 장치 바인드 및 연결 (는 BUSID 에서 list 확인)
usbipd bind --busid <BUSID>
usbipd attach --wsl --busid <BUSID>
```

4단계: WSL2에서 포트 확인

```
ls /dev/ttyACM* # 또는 ls /dev/ttyUSB*
lerobot-find-port
```

- USB 장치를 분리했다가 다시 연결하면 usbipd attach를 다시 실행해야 한다
- Leader 팔과 Follower 팔 컨트롤러 모두 WSL2에 연결해야 한다
- 카메라(웹캠)도 동일한 방식으로 연결한다

6.3 Step 2: 모터 ID 설정 (최초 1회)

모터 ID 설정

```
# Follower 팔 실제 (작업 수행)
lerobot-setup-motors \
  --robot.type=koch_follower \
  --robot.port=/dev/ttyACM1

# Leader 팔 텔레오퍼레이션용()
lerobot-setup-motors \
  --teleop.type=koch_leader \
  --teleop.port=/dev/ttyACM0
```

6.4 Step 3: 캘리브레이션

캘리브레이션

```
# Follower 캘리브레이션
lerobot-calibrate \
  --robot.type=koch_follower \
  --robot.port=/dev/ttyACM1 \
  --robot.id=my_koch_follower

# Leader 캘리브레이션
lerobot-calibrate \
  --teleop.type=koch_leader \
  --teleop.port=/dev/ttyACM0 \
  --teleop.id=my_koch_leader
```

6.5 Step 4: 텔레오퍼레이션 테스트

텔레오퍼레이션 테스트

```
lerobot-teleoperate \
```

```
--robot.type=koch_follower \  
--robot.port=/dev/ttyACM1 \  
--robot.id=my_koch_follower \  
--robot.cameras="{ front: {type: opencv, index_or_path: 0, \  
    width: 640, height: 480, fps: 30}}" \  
--teleop.type=koch_leader \  
--teleop.port=/dev/ttyACM0 \  
--teleop.id=my_koch_leader \  
--display_data=true
```

확인사항

- Leader 팔을 움직이면 Follower가 실시간으로 따라 움직이는가?
- 카메라 영상이 화면에 정상적으로 표시되는가?
- 움직임이 부드럽고 떨림이 없는가?

7 데이터 수집 및 Fine-tuning

7.1 시연 데이터 수집

사람이 Leader 팔을 조종하여 작업을 시연하는 데이터를 녹화한다.

7.1.1 CLI로 수집

시연 데이터 녹화 (50 에피소드)

```
HF_USER=$(huggingface-cli whoami | head -1)

lerobot-record \
  --robot.type=koch_follower \
  --robot.port=/dev/ttyACM1 \
  --robot.id=my_koch_follower \
  --robot.cameras="{ front: {type: opencv, index_or_path: 0, \
    width: 640, height: 480, fps: 30}}" \
  --teleop.type=koch_leader \
  --teleop.port=/dev/ttyACM0 \
  --teleop.id=my_koch_leader \
  --display_data=true \
  --dataset.repo_id=${HF_USER}/koch_pick_place \
  --dataset.num_episodes=50 \
  --dataset.single_task="Pick up the block and place it in the bin" \
  --dataset.streaming_encoding=true
```

7.1.2 Python API로 수집

src/05_collect_data.py

데이터 수집 팁

- Enter: 다음 에피소드 시작 Backspace: 현재 에피소드 삭제
- 최소 **50개 에피소드** 권장 (많을수록 좋다)
- 다양한 시작 위치와 물체 배치를 사용한다
- 일정한 속도로 부드럽게 움직인다

7.2 SmolVLA Fine-tuning

7.2.1 CLI로 Fine-tuning (권장)

SmolVLA Fine-tuning

```
HF_USER=$(huggingface-cli whoami | head -1)

lerobot-train \
  --policy.path=lerobot/smolvla_base \
  --dataset.repo_id=${HF_USER}/koch_pick_place \
  --output_dir=outputs/train/smolvla_koch \
  --job_name=smolvla_koch_pick_place \
  --policy.device=cuda \
  --policy.dtype=bfloat16 \
```

```
--batch_size=4 \
--steps=100000
```

7.2.2 Python API로 Fine-tuning

```
src/06_finetune_smolvla.py
```

7.3 주요 설정 파라미터

Table 7: SmolVLA 핵심 설정 파라미터

파라미터	기본값	설명
chunk_size	50	한 번에 예측하는 action step 수
n_action_steps	50	재추론 전까지 실행할 action 수
num_steps	10	Flow matching denoising 스텝 수
freeze_vision_encoder	True	Vision encoder 동결 여부
train_expert_only	True	Action expert만 학습 여부
optimizer_lr	1e-4	학습률
gradient_checkpointing	False	메모리 절약 (True 권장)

GPU 메모리 부족 시

```
--batch_size=2 \
--policy.gradient_checkpointing=true \
--policy.freeze_vision_encoder=true \
--policy.train_expert_only=true
```

8 추론 및 평가

학습이 완료된 모델로 Koch v1.1을 자율 제어한다.

8.1 CLI로 실행

학습된 SmolVLA로 로봇 자율 제어

```
HF_USER=$(huggingface-cli whoami | head -1)

lerobot-record \
  --robot.type=koch_follower \
  --robot.port=/dev/ttyACM1 \
  --robot.id=my_koch_follower \
  --robot.cameras="{ front: {type: opencv, index_or_path: 0, \
    width: 640, height: 480, fps: 30}}" \
  --display_data=true \
  --dataset.repo_id=${HF_USER}/eval_smolvla_koch \
  --dataset.single_task="Pick up the block and place it in the bin" \
  --policy.path=${HF_USER}/smolvla_koch_pick_place
```

실행 시 주의사항

- Leader 팔은 **연결하지 않는다** (정책이 직접 행동을 생성)
- 로봇 주변에 안전 공간을 확보한다
- 비상 정지: Ctrl+C

8.2 Python API로 실행

```
src/07_run_policy_koch.py
```

이 스크립트는 자율 제어 후 각 에피소드의 성공 여부를 수동으로 기록하여 성공률을 계산한다.

9 전체 워크플로우 요약

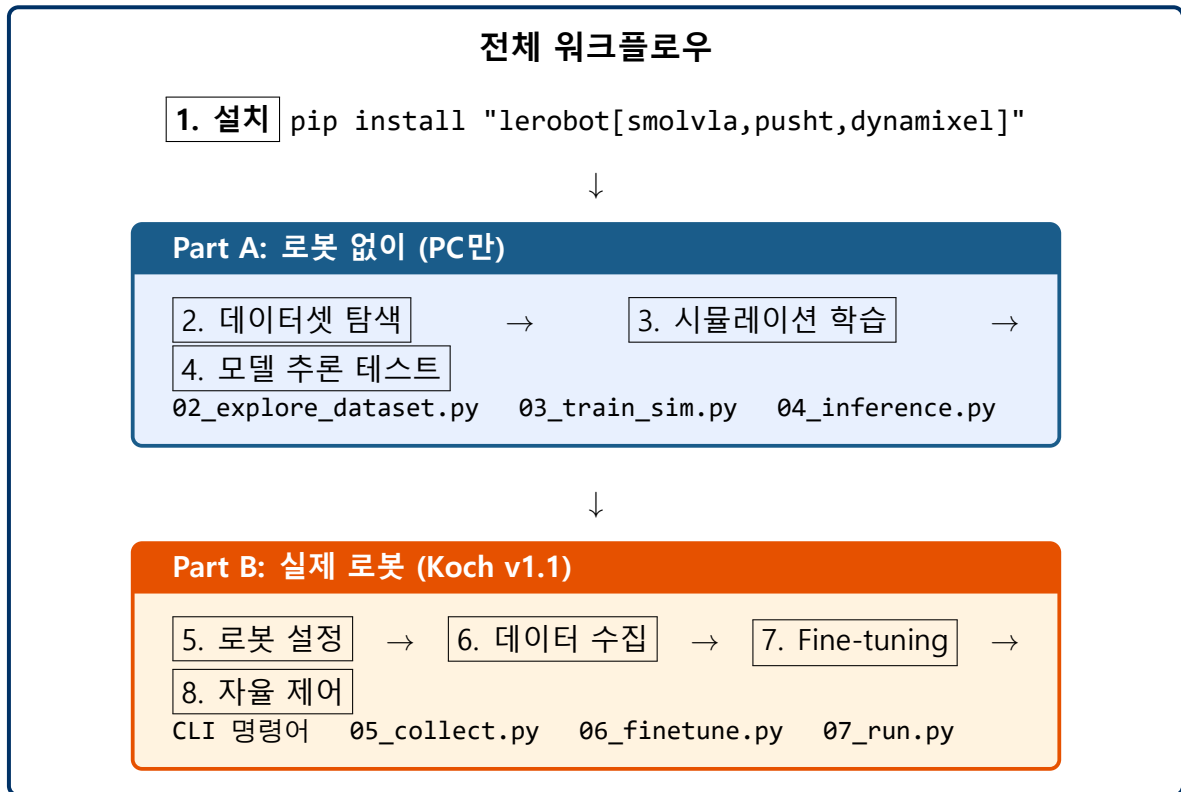


Figure 2: Part A (로봇 불필요) → Part B (로봇 필요) 전체 흐름

9.1 소스코드 파일 목록

Table 8: 제공되는 소스코드 파일 (src/ 폴더)

파일명	용도	로봇 필요	설명
<code>01_test_install.py</code>	설치 확인	아니오	패키지, GPU 확인
<code>02_explore_dataset.py</code>	데이터 탐색	아니오	공개 데이터셋 구조 파악
<code>03_train_sim_diffusion.py</code>	시뮬레이션 학습	아니오	PushT에서 Diffusion 학습
<code>04_inference_no_robot.py</code>	추론 테스트	아니오	데이터셋 프레임으로 예측
<code>05_collect_data.py</code>	데이터 수집	예	텔레옴으로 시연 녹화
<code>06_finetune_smolvla.py</code>	Fine-tuning	아니오*	SmolVLA 모델 학습
<code>07_run_policy_koch.py</code>	자율 제어	예	학습 모델로 로봇 실행

*06번은 수집된 데이터셋이 Hub에 있으면 로봇 없이 실행 가능

10 트러블슈팅

Table 9: 자주 발생하는 문제와 해결 방법

문제	해결 방법
Permission denied: /dev/ttyACM*	sudo usermod -aG dialout \$USER 후 재로그인
CUDA out of memory	batch_size 줄이기, gradient_checkpointing=true, train_expert_only=true
카메라가 인식되지 않음	ls /dev/video*로 확인, index_or_path 값 변경
모터가 응답하지 않음	전원 공급 확인, USB 연결 확인, lerobot-find-port 재실행
캘리브레이션 실패	모터를 수동으로 전체 범위 움직인 후 재시도
학습 loss가 감소하지 않음	데이터 품질 확인, learning rate 조절 (5e-5 시도)
추론 시 로봇이 떨림	num_steps 증가 (10→20), 데이터 추가 수집
ffmpeg not found	sudo apt install ffmpeg (WSL2/Linux), Windows: https://ffmpeg.org 에서 다운로드 후 PATH에 추가
ModuleNotFoundError: gym_pusht	pip install "lerobot[pusht]" 으로 재설치
— Windows 11 / WSL2 관련 —	
WSL2에서 nvidia-smi 실패	Windows 측 NVIDIA 드라이버를 최신 버전으로 업데이트 (525.60+). WSL2 내부에 드라이버를 설치하지 않는다
WSL2에서 USB 장치 미인식	usbipd-win을 설치하고 usbipd attach --wsl --busid <ID>로 연결한다
WSL2에서 GUI 창이 안 뜸	wsl --update로 WSLg 업데이트, Windows 11 22H2 이상 필요
WSL2에서 카메라 미인식	usbipd로 카메라 USB를 WSL2에 연결 후 ls /dev/video*로 확인
Windows 네이티브에서 gym-pusht 설치 실패	WSL2를 사용하거나 Google Colab에서 실습한다

11 참고 자료

자료	URL
LeRobot GitHub	https://github.com/huggingface/lerobot
SmolVLA 논문	https://arxiv.org/abs/2506.01844
SmolVLA Base 모델	https://huggingface.co/lerobot/smolvla_base
VLM Backbone	https://huggingface.co/HuggingFaceTB/SmolVLM2-500M-Video-Instruct
Koch v1.1 하드웨어	https://github.com/jess-moss/koch-v1-1
LeRobot 문서	https://huggingface.co/docs/lerobot/index
Koch 설정 가이드	https://huggingface.co/docs/lerobot/koch
SmolVLA Colab 노트북	https://colab.research.google.com/github/huggingface/notebooks/blob/main/lerobot/training-smolvla.ipynb
Robot Learning Tutorial	https://huggingface.co/spaces/lerobot/robot-learning-tutorial

11.1 인용

BibTeX

```
@article{shukor2025smolvla,  
  title = {SmolVLA: A Vision-Language-Action Model for  
          Affordable and Efficient Robotics},  
  author = {Shukor, Mustafa and Aubakirova, Dana and  
           Capuano, Francesco and others},  
  journal = {arXiv preprint arXiv:2506.01844},  
  year = {2025}  
}
```